Cryptographically Enforced Delegated Spending Protocol

Anjul kc

November 2026

Introduction

Delegation, in its simplest form, is the act of granting another party authority to perform certain operations under a policy defined by the delegator. One of the elegant applications of cryptography in this domain is the *Proxy Signature*, which allows an original signer (the delegator) to grant another party (the proxy signer) the ability to sign messages on their behalf.

Depending on the construction, delegation can be achieved through several approaches:

1. **Private-key disclosure:** The delegator reveals their entire private key to the proxy signer, effectively transferring unrestricted signing authority.
2. **Private-key derivation:** The delegator uses their private key to derive a separate delegation key, which is then provided to the proxy signer.
3. **Policy-based delegation:** The delegator grants signing authority subject to predefined restrictions or policies.

The latter two approaches are particularly powerful because they allow the delegator to retain control over their original private key while restricting the authority granted to the proxy signer. Depending on the construction, these restrictions may include limitations on the messages that can be signed, the scope of authority, or the duration of delegation. Furthermore, revocation mechanisms can be incorporated to invalidate previously granted delegation rights.

The Problem of Consumption

When examining anonymous cryptocurrencies and token allowance protocols, we can identify an interesting limitation shared by many constructions: the consumption of delegated or issued authorization is often indivisible or restricted to one-time use.

For example, in anonymous cryptocurrency systems, a *nullifier* is typically used to prevent a deposited note or coin from being spent more than once. Once the corresponding nullifier is published, the associated spending authorization is considered consumed. Similarly, one-time authorization mechanisms, such as signatures used in *Permit2*, can be designed to prevent replay by ensuring that an authorization cannot be reused.

However, this behavior is closely tied to the objectives and security requirements of each protocol. Anonymous cryptocurrencies primarily focus on enabling private transfers, deposits, and withdrawals while preventing double-spending. Permit2, on the other hand, focuses on allowing users to authorize smart contracts to access and transfer their tokens under specified conditions.

Although these mechanisms serve their respective purposes effectively, they do not necessarily provide a general mechanism for *partially consuming delegated authority while preserving the remaining authorization for future use.*

Objective

Partial consumption and transparency are the fundamental principles of this protocol. The term *spending* is not limited to monetary transactions but encompasses any resource or entitlement that can be consumed incrementally.

The protocol leverages decentralized infrastructure for cryptographic verification and state management, while relying on a central authority to issue and manage allowances. This hybrid architecture combines decentralized enforcement and verifiability with centralized allowance administration.

High level Idea

Suppose Alice deposits 10 ETH into protocol contract. She defines three independent spending policies on behalf of Bob, John, and Smith:

- Bob is authorized to spend up to 4 ETH.
- John is authorized to spend up to 4 ETH.
- Smith is authorized to spend up to 2 ETH.

Alice generates three distinct secret tokens and distributes one to each of them, allowing them to consume their respective allowances however they wish.

Revealing or consuming a secret token would be straightforward if the entire allowance were spent in a single transaction. However, DSP adopts the concept of *proxy signatures*, allowing each authorization to be exercised through multiple transactions.

Consider Bob, who owns two Ethereum accounts. He decides to transfer 1 ETH to his secondary account and 3 ETH to his primary account. He can do so because the authorization is associated with his secret token and spending policy, rather than being restricted to a particular Ethereum account.

John, meanwhile, withdraws only 3 ETH from his 4 ETH allowance, leaving 1 ETH unconsumed.

Unfortunately, Smith loses his secret token before using it. Since the token represents his delegated spending authority, he contacts Alice and requests a replacement. Alice revokes the previous authorization and prepares a new token for Smith, allowing him to continue under the same spending policy.

Smith is happy to receive his replacement and eventually withdraws his full 2 ETH allowance.

After some days, John decides not to use his remaining 1 ETH. Since this amount was never consumed, it remains available to Alice under the protocol's withdrawal policy. Alice can therefore reclaim the remaining 1 ETH.

Ultimately, Bob consumes 4 ETH, Smith consumes 2 ETH, John consumes 3 ETH, and Alice reclaims the remaining 1 ETH.

Adoption in Next-Generation Digital Wallets

Another potential application of the Delegated Spending Protocol (DSP) is the development of next-generation digital wallets. At first, the idea of building a digital wallet on top of an existing wallet may sound unusual. However, the proposed architecture follows a concept similar to the integration of traditional e-banking services with digital wallets, where users load funds into a wallet and subsequently spend them through its interface.

DSP introduces a different approach to this integration. Instead of transferring funds into the digital wallet, the spender generates a cryptographic spending token and transfers it to the digital wallet. This token represents a predefined spending authority, allowing the digital wallet to initiate payments on behalf of the spender without necessarily taking custody of the underlying funds.

When a payment is initiated, the digital wallet presents the token and the corresponding authorization to the underlying spending mechanism. The protocol verifies the authorization, checks the applicable spending policy and remaining allowance, and permits the transaction only if the conditions are satisfied.

This approach separates the custody of funds from the authority to spend them. The user's original wallet retains control over the funds, while the digital wallet operates as an interface through which delegated spending permissions can be exercised.

Consequently, DSP could enable digital wallets to facilitate payments without requiring users to deposit or transfer their funds into the digital wallet beforehand, Rather authorize transfer of balance to other wallet address.

It solves the following:

- Non-technical users: They don't need to understand blockchain transactions, gas fees, private keys, or wallet interfaces. They interact with a familiar digital wallet.

- Technical users: They don't need to unlock their primary wallet and manually authorize every payment. They can delegate a bounded spending allowance to their digital wallet, Reducing private-key exposure.

Similarly there are numerous application such as allocation of budget, subscription based protocol, parental control spending. Automated agents -: An AI agent or automated service receives permission to pay for APIs computing resource or service upto a defined budget.

Technology Overview

Proxy signatures are at the core of our implementation. Although the construction follows the general principles of Schnorr signatures, it requires several customizations to accommodate the requirements of our protocol. These include message consistency, separation of on-chain and off-chain computation, and, most importantly, protection against replay attacks.

In conventional one-time signature schemes, a unique signature and its associated nonce are sufficient for authorizing a single resource consumption, provided that the protocol correctly enforces one-time usage. However, our protocol supports incremental and repeated consumption of delegated resources. Consequently, a monotonically increasing counter provides a more suitable replay-protection mechanism. The smart contract maintains this counter and uses its current value to establish the next signing challenge.

The construction will be demonstrated using the BLS12-381 elliptic curve. This choice is motivated by the availability of dedicated Ethereum precompiles for BLS12-381 operations, whereas performing the required native secp256k1 elliptic-curve operations directly within the Ethereum Virtual Machine can be computationally expensive.

This construction serves as the general cryptographic framework for delegated authorization and spending within our protocol.